

PROTOCOL WHITEPAPER

Dynamo

The Interplanetary AI Economic and Finance Layer

One enforceable budget. Many agents.

Verified-value payments - stream payment, across the universe.

Dynamo Protocol Team

Version 2.1 · September 2026

dynamo.zone

Contents

Abstract	3
1. Introduction	4
2. Background and Related Work	5
3. Design Doctrines	7
4. System Model and Architecture	8
5. The Core Protocol, Formally	10
6. Security Properties and Analysis	19
7. Implementation and Evaluation	21
8. The Grand Vision: From Protocol to the Finance and Economic Layer of the AI Agent Economy	26
9. The DYNAMO Token (Design Space)	29
10. Discussion	33
11. Conclusion	34
References	35
Appendix A - Notation	38
Appendix B - Demonstrated Behaviors	39
Appendix C - Implementation Scope and Reproducibility	40

Abstract

Autonomous software can incur costs across multiple services before a human reviews its activity. Dynamo is a metering and settlement protocol that binds delegated spending authority to explicit accounting limits and funding-specific authorization. A parent budget allocates capped streams to service nodes, including delegated subtrees; metered work produces cumulative signed payment vouchers; and settlement is constrained by stream, ancestor, and aggregate caps. We formalize these conservation properties and derive replay-safety, rounding, asset-conversion, and checkpoint-cadence results under stated assumptions. We distinguish settlement safety from end-to-end control of external expenditure, and authenticated observations from proof of delivery. The escrow implementation supports rate-envelope clipping and payee-funded breakpoint refunds; a solvency-independent, delivery-conditioned holdback is a separate proposed mechanism. The architecture combines a no-money control ledger, on-chain escrow, and a card-authorization backend through a common object model while preserving each rail's distinct operational and finality assumptions. Public contracts, client packages, conformance tooling, and reported demonstrations provide the implementation and evaluation basis. Finally, we examine Dynamo as a prospective finance and economic layer for the AI agent economy, including conditional token, dedicated-ledger, and interplanetary extensions. The contribution is the composition of bounded delegation, metering, and auditable settlement, rather than novelty of the individual cryptographic or financial primitives.

Keywords: AI agents; delegated spending; payment channels; streaming settlement; budget enforcement; cumulative vouchers; evidence; stablecoins; mechanism design.

1. Introduction

Dynamo coordinates spending across workflows that use multiple paid services. A budget owner defines an aggregate ceiling, allocates capped streams to providers, and authorizes payment through cryptographically signed cumulative vouchers. A provider can delegate part of its stream’s capacity to other providers, creating a tree of bounded spending authority. An orchestrator can therefore account for several agents, APIs, or human contributors under one budget. Control Mode exercises this accounting without transferring value; the escrow and card backends connect it to funding and settlement.

Software increasingly participates in economic workflows by invoking paid APIs, orchestrating metered services, and purchasing intermediate outputs. Research on the agentic economy identifies programmatic consumer-service interactions as a possible source of changes in market structure [10]. METR’s 2025 study reports an approximately seven-month doubling time in the human-duration tasks that tested agents could complete with 50% success; this measures a particular capability, not commercial adoption or autonomous financial reliability [14]. Gartner’s prediction that more than 40% of agentic-AI projects will be canceled by the end of 2027 is a forecast, not an observed cancellation rate [15]. Empirical work on multi-agent failures identifies design, coordination, and verification failures that motivate explicit operational controls [21]. These findings motivate the problem without establishing demand for any particular protocol.

A budget dashboard, an admission controller, an authorization mandate, and a settlement contract address different questions. A dashboard estimates or records expenditure; an admission controller can reject prospective work; a mandate specifies delegated authority; a settlement contract constrains transfers. Some systems combine several of these functions. Current x402 documentation includes maximum-amount payments, batch settlement, and signed offers and receipts; MPP supports programmatic payments and session-based use; AP2 standardizes authorization and transaction evidence [18, 23-27]. Dynamo focuses on coordinating these concerns around a shared, recursively allocated budget.

Dynamo’s organizing proposition is that a workflow’s spending policy should be reflected in its settlement authority. A funded budget is divided into capped streams; metering generates signed cumulative claims; the backend admits only claims consistent with its rules. On the escrow rail, this can make *settlement above the accounting cap* impossible under the modeled contract transitions. Preventing *all upstream expenditure* requires more: every billable request must pass a non-bypassable admission boundary, uncertain costs must be bounded or reserved before execution, and concurrent actors must not independently consume the same headroom. An agent retaining an unrestricted provider credential can incur costs outside Dynamo regardless of the vault’s correctness.

The intended application is broader than language-model tokens. Service adapters can price compute time, API calls, bytes, rendered frames, energy, or human work. The engine uses a canonical accounting convention of one accounting unit per micro-USD. Payload neutrality means that service-specific measurement and pricing occur outside the settlement kernel; it does not mean that raw kilowatt-hours and raw tokens are numerically interchangeable. Section 5 defines the units and their conversion into payment claims.

Contributions.

1. **A compositional budget model** (Sections 4 and 5.1-5.3): a parent ceiling and delegated stream caps, with explicit assumptions for node, subtree, and aggregate conservation. Nesting depth is bounded at eight; a general service graph must be represented by one or more allocation trees.

2. **A cumulative-voucher settlement model** (Section 5.5): monotone accounting state gives a telescoping no-double-payment result without a per-voucher nonce. Domain separation and authenticated stream bindings remain essential; allowance credentials use their own nonces.
3. **A scoped verified-value design** (Section 5.8): signed observations, breach policies, breakpoints, and refund accounting are separated from universal delivery verification and from reserve-backed recovery after payment.
4. **A common funding abstraction** (Section 4.2): shared budget and stream objects across control, escrow, and card backends, with explicit custody relationships, settlement granularity, reversibility, and finality.
5. **An implementation and evaluation framework** (Section 7): public contracts and client artifacts, reported testing and demonstrations, and reproducibility criteria that distinguish configured campaigns, execution results, and independent replication.
6. **An economic-layer research agenda** (Sections 8 and 9): a path from spend control to multi-party settlement, including conditional governance, trust-bond, token, dedicated-ledger, and interplanetary proposals. Their adoption and viability require evidence beyond the cap invariant.

2. Background and Related Work

Off-chain payment channels. Bitcoin and Ethereum provide the underlying ledger models [1, 2]. Lightning uses signed off-chain updates with later on-chain enforcement, and payment-channel-hub work such as BlindHub studies conditional multi-party payments and privacy [3, 22]. Dynamo adopts the established idea of settling cumulative signed state. Its emphasis is a shared budget represented as an allowance tree, rather than a claim to have invented multi-party payments. Unlike a mature payment-channel protocol with a dispute period, the vault permits a budget owner to revoke and close before an outstanding voucher is submitted; this payer-safety/payee-liveness trade-off matters in any comparison.

Streaming money. Sablier provides token-streaming primitives [20]. Dynamo combines a cumulative rate envelope with metered claims: wall-clock time bounds what may settle, but time alone does not create the next payable voucher. A service adapter determines what constitutes billable work. The distinction is between an authorization envelope and a delivery-dependent accounting process, not between protocols that do and do not support time.

Agent payment standards. The 2025 x402 whitepaper defines the initial HTTP payment design [9]. The September 2026 documentation describes additional schemes and extensions. Its upto scheme authorizes a maximum and settles an amount determined by actual usage; batch settlement and signed offers/receipts expand the evidence and efficiency surfaces [23-26]. MPP, introduced by Stripe and Tempo in March 2026, coordinates machine payments across methods and supports microtransactions and recurring use; the public MPP site also describes an evolved sessions implementation [18]. AP2 v0.2 binds checkout and payment mandates to receipts and supports human-present and autonomous modes. It discusses agent-to-agent delegation conceptually while leaving it outside the specification [27]. Dynamo’s proposed contribution is a particular composition of recursive allocation, accounting caps, and evidence-linked settlement. Interoperability with a standard must be demonstrated against a named version and scheme; an HTTP 402 response alone is not sufficient.

Identity and validation. ERC-8004 specifies identity, reputation, and validation registries [13]. Xiong et al.’s empirical study covers Ethereum, BSC, and Base through 13 May 2026 and reports substantial weaknesses in the sampled identity and reputation records [12]. This is evidence about a dated sample, not a timeless verdict on all registry deployments. Vaziry et al. integrate ledger-anchored agent identities with A2A and x402

payments [11]. Dynamo may complement these approaches by attaching settlement records to interactions, but paid interactions can themselves be manufactured or collusively recycled; payment evidence is not automatically honest reputation.

Chargebacks and the evidence gap. Agent-initiated commerce raises distinct questions of authority, delivery, and liability. AP2 includes dispute evidence, and Visa’s Trusted Agent Protocol specifies signed agent requests, intent, and related recognition and payment information [27, 28]. Mastercard’s Verifiable Intent is a complementary authorization initiative [29]. Dynamo’s opportunity is to record metered service performance and connect it to financial state. Whether a particular evidence bundle satisfies a dispute process depends on that process, jurisdiction, contractual terms, and externally anchored identities. Cryptographic integrity alone neither establishes legal admissibility nor allocates liability.

The counter-commoditization problem. Basic budget counters and quotas can be implemented in gateways and observability tools. Their enforceability depends on architecture, not category labels. Dynamo seeks differentiation through the coupling of those controls to settlement and a shared multi-service budget. This is a product and systems hypothesis requiring comparison against well-engineered alternatives, including transactional gateway counters and reserved-spend implementations; it is not a theorem that competitors cannot add settlement.

Agent tooling. MCP standardizes tool integration [17]. Dynamo’s monetize() surface and gateway adapters address service monetization and spend control. They allow tool-specific measurement and pricing to be connected to a common payment interface. The extent of unmet monetization demand is a market question, not an implication of protocol compatibility.

Fee markets and transaction ordering. EIP-1559 analysis and research on miner/maximal extractable value provide relevant economic context [7, 8]. Signed vouchers constrain a claim’s contents, but do not remove transaction-ordering effects. Inclusion delays, fee changes before settlement, oracle observations, and revoke/close ordering can affect outcomes. Dynamo is not MEV-free merely because it uses signatures.

Oracles. Chainlink motivates authenticated external price inputs [6]. Dynamo’s adapter checks freshness and deviation relative to a stored baseline. These checks constrain accepted observations; they do not independently establish a correct market price, eliminate a compromised feed, or guarantee that collateral remains sufficient after a price move (Section 5.9).

Token design. Hyperliquid provides an example of automated fee conversion into a native token and native-token gas usage. Its documentation describes multiple fee destinations and assistance-fund burning [31, 32]. Render provides a different burn-and-emission mechanism [33]. These precedents inform the design questions in Section 9; they do not establish a transferable buyback optimum, launch threshold, or valuation multiple.

Table 1. Comparison of documented functions, not an exhaustive market survey.

System	Principal scope	Spending control and evidence	Distinction relevant to Dynamo
x402 [23-26]	HTTP payment negotiation and schemes	Exact and capped variable payments; batch settlement; signed offers/receipts	A recursive aggregate-budget policy is a separate composition question, not an absence of payment controls
MPP [18]	Machine payments across methods, including sessions	Method-specific authorization and settlement; evolving session tooling	Compare a named session method and implementation, not only bilateral request syntax

System	Principal scope	Spending control and evidence	Distinction relevant to Dynamo
AP2 v0.2 [27]	Agent authority and transaction evidence	Constrained mandates, checkout/payment receipts	Authorization evidence is complementary to metered performance and funding state
ERC-8004 [12, 13]	Identity, reputation, validation	Registry-based claims; deployment-specific verification quality	Does not by itself provide Dynamo's budget conservation rule
Dynamo	Budget-tree metering and settlement	Contract accounting caps; cumulative vouchers; additional engine policies	Directly implemented escrow rules are narrower than the complete verified-value design

3. Design Doctrines

Six doctrines guide the architecture. Each specifies a design constraint together with the boundary at which it can be enforced.

D1 – Non-custodial operation. The protocol service is designed not to operate an omnibus user balance or perform fiat/crypto conversion. The vault administrator has no arbitrary principal-withdrawal method, but may change the fee within a 10% bound and change its collector. The budget owner has broader rights, including signing vouchers and closing budgets. A hosted operator may control that owner's key; an application treasury may receive end-user payments; a card PSP controls parts of the payment process. Contract restrictions, key custody, application custody, and legal custody are therefore separate analyses. The four constraints are no arbitrary administrative withdrawal, redirection, freezing, or general stored-value account. They do not remove external token-issuer powers or make an operator-held key self-custody [36, 37].

D2 – Fail closed wherever money moves. Invalid or unverifiable authorization should prevent a new financial commitment. This is a safety policy, not a promise of availability or recovery. A rejected transaction does not undo work already performed upstream, reverse a previously finalized transfer, or automatically reconcile a partially completed cross-system operation. Fail-closed behavior must be specified per transition, including crash boundaries and retries.

D3 – Objective facts only. Settlement policies may use measurable quantities such as durations, unit counts, error rates, availability, or an explicitly recorded acceptance decision. A signature proves that a key attested to a statement; it does not prove the statement true. Percentiles require a specified population, sampling method, window, and clock. Explicit acceptance records a decision, not necessarily objective product quality. Subjective quality scores are excluded from the settlement kernel; richer quality markets require a separately specified validation layer.

D4 – Payload neutrality. The kernel is independent of prompts, media, or service content. Adapters map service measurements into canonical accounting units. They remain responsible for pricing correctness, measurement provenance, and any delivery predicate. A payload-free financial record can still reveal commercially sensitive metadata.

D5 – Mode identity. Budgets, streams, and cumulative claims share an interface across funding backends. Identity means an intended common object model and conformance obligations, not identical economic outcomes. A no-money ledger has no solvency risk; an ERC-20 transfer has chain-specific finality; a card authorization can expire and captures can be disputed. These differences must remain explicit (Section 4.2).

D6 – The data boundary. The metering interface is designed to transmit settlement facts rather than service payloads. Offline credential verification can avoid a remote verification request in the gateway path; self-hosting and opt-in telemetry support this boundary. Identifiers, amounts, timing, public-chain linkability, logs, and compromised adapters remain privacy risks even when prompts and outputs do not enter the financial record. They are part of the threat model [39].

4. System Model and Architecture

4.1 Parties and layers

The parties are a **budget owner** authorizing expenditure, **service nodes** receiving settlement, an optional **oracle role** attesting breakpoints, a **fee collector**, and an **operator** running a core instance. An end user, an application treasury, the budget owner, and the protocol administrator need not be the same entity.

L0 provides funding, authorization, and rail-specific settlement. **L2** is the stream engine providing budgeting, delegation, metering, signing, and checkpointing. These are component labels; the present architecture does not require a separate Dynamo L1. The backend abstraction includes funding, funded-ceiling queries, settlement, refunds, and release. It isolates implementation concerns while preserving rail-specific risk.

Enforcement occurs at two points. The engine checks prospective accounting changes; the escrow contract independently checks the subset of rules encoded in its settlement function. The latter includes authenticated voucher authority, monotonicity, time bounds, the stream’s rate envelope, ancestor caps, the budget cap, and sufficient asset balance. It does **not** evaluate every engine anomaly, SLA, exposure, or concurrency predicate. Re-implementation creates defense in depth and a need for differential testing; it does not make divergence logically impossible [36].

4.2 Three funding modes, one object graph

Mode	Ceiling	Settlement operation	Distinct assumptions
Control (none)	Local accounting allocation	Durable ledger entry, no value transfer	A simulated funding constraint; no proof of money backing
Escrow (usdc)	Deposit in the configured vault, with an accounting cap	Accepted voucher triggers ERC-20 transfers	Token behavior, chain execution, keys, gas, and collateral valuation
Card (card)	PSP authorization for a bounded amount	Captures and connected-account transfers	PSP capability/approval, authorization lifetime, cent granularity, disputes

Changing the funding selector can preserve application-level object shapes, but production migration also requires keys, asset and chain configuration, PSP onboarding where applicable, and error/finality handling. The published client, `@dynamoprotocol/sdk 2.1.0`, is a thin HTTP interface rather than an embedded engine. Engine capabilities must be distinguished from operations exposed by that client [39, 42].

Conformance criterion. For a common supported operation, compare normalized preconditions, accounting state transitions, and error categories; separately document permissible rail differences. A successful Control Mode test is useful but cannot validate an ERC-20 transfer or a PSP’s production acceptance. Reusing a schema is not an observational-equivalence proof.

4.3 Threat model (summary)

Adversaries may control agents or service nodes; obtain a scoped credential; replay or reorder messages; withhold a result; collude with an attester; or exploit delayed observations. Hosts may crash, clocks may disagree, networks may partition, chains may reorganize, and a PSP may decline or time out. Administrators, budget-owner signers, application treasuries, token issuers, and measurement sources have distinct powers and cannot be merged into one generic trusted role.

The contract results in Section 5 assume: (A1) atomic execution and rollback of the modeled EVM transitions; (A2) unforgeable signatures and correctly bound keys/domains, except for explicitly compromised authorized keys; (A3) the non-upgradeable contract behavior at the cited source snapshot and valid arithmetic ranges; (A4) settlement assets satisfying the modeled transfer semantics, without unmodeled rebases or balance seizures; and (A5) observations of the relevant canonical chain state. End-to-end spending control additionally requires (A6) non-bypassable admission and atomic reservations or disjoint preallocated authority. Provider payment and availability require further liveness assumptions and are not implied by A1-A6.

4.4 The request path and durability

Two request paths must be distinguished. In the **owner-API path**, the HTTP SDK calls the core; the engine checks a metering request, records it, signs a voucher when appropriate, and checkpoints through the backend. This path incurs network and processing latency. In the **gateway path**, allowance credentials can be verified locally before forwarding service traffic, with subsequent metering reports. Local verification removes a particular remote call, not all computational overhead, and does not by itself synchronize a shared budget across replicas [39].

The engine uses file-backed SQLite WAL journaling, synchronous = FULL, and a pinned schema version. Recovery discards consume records not covered by a signed voucher and releases held reservations, establishing a local billing frontier. These mechanisms do not make a transaction spanning SQLite, an EVM chain, and a PSP atomic. Backend success can precede the local record write, while the revocation path commits locally before calling the backend. The hosted runtime also has a budget-rehydration limitation after restart. A complete recovery procedure must address each of these boundaries rather than relying only on database durability.

Thus, “billing truth is the last signed voucher” means the intended durable local entitlement frontier, not a complete cross-system recovery algorithm. An independently observed backend transfer remains a financial fact even when its checkpoint row is missing. A production recovery design needs durable operation identifiers, queryable backend outcomes, idempotent retry, and reconciliation before reopening spending authority. These are acceptance requirements where the current implementation does not yet supply them.

4.5 The accounts model

The escrow deployment has five operational roles: the **deployer** owns initial administration; the **core owner** funds and closes budgets; the **service node** submits settlements and needs gas; the **application treasury** collects end-user payments and issues user refunds; and the **fee collector** receives fees. Roles may share addresses, but sharing changes compromise boundaries. Monitoring must distinguish token balances from gas balances.

For a job settling gross accounting amount D in a six-decimal identity-priced asset, the core owner funds the escrow and receives only its unspent remainder; the service node receives D less the fee; the collector receives the fee. An application that collected the user’s payment holds a separate obligation. Closing the vault returns principal to the recorded budget owner, **not automatically to the application’s end user**. The

application's refund is a separate transfer, and its solvency and reliable outbox are separate responsibilities [36, closeBudget].

Combining budget-owner and payee addresses can reduce treasury drift in a self-operated test, but also combines their compromise boundaries. Such a configuration does not demonstrate settlement between independent counterparties. Evaluation must identify which roles are independently controlled, and applications must monitor the funding account separately from the account that collects end-user payments.

5. The Core Protocol, Formally

5.1 The budget tree

Let a budget B have accounting cap $C \in \mathbb{N}_{>0}$. Amounts are nonnegative integers in micro-USD accounting units, with 10^6 units per USD denomination. This is an accounting convention; a token's actual purchasing power may differ. All divisions in the financial kernel require a specified rounding direction. The implementation uses integer arithmetic; overflow must reject rather than wrap [36, 37].

A stream is represented by

$$s = (\text{node}_s, r_s, c_s, t_0(s), \rho_s, d_s), \quad (1)$$

where $r_s > 0$ is an accounting-unit-per-second envelope, $c_s > 0$ is a subtree cap, $t_0(s)$ is the backend start time, ρ_s is a revocation timestamp or $\perp$, and $1 \leq d_s \leq 8$. A budget has a forest of root streams; adjoining a virtual budget root gives a tree. A service dependency graph with joins or cycles is not literally this tree: its obligations need an unambiguous allocation mapping to avoid double counting.

Opening a root requires $\sum_{i \in \text{roots}(B)} c_i \leq C$. Opening a child of p requires

$$c_s + \sum_{j \in \text{existingChildren}(p)} c_j \leq c_p. \quad (2)$$

Allocations are monotone: revocation does not return the allocated cap. This avoids reusing authority that may still be associated with outstanding claims, but strands capacity and can exhaust allocatable headroom without exhausting funds. It is one conservative implementation choice, not the only safe allocation design. A child allocation also is not a guarantee that its full cap remains spendable: the parent's own payments share the same subtree cap [36, _openStream, settle].

5.2 The cap invariant

Write S_s for stream s 's **gross cumulative settled accounting amount**, $\mathcal{T}(s)$ for its subtree, and

$$Q_s = \sum_{u \in \mathcal{T}(s)} S_u, \quad T_B = \sum_{u \in B} S_u. \quad (3)$$

Refunds do not decrement these gross counters in the contract. Accordingly, they do not restore consumed cap capacity. The relevant safety statement is

$$\forall s : \quad 0 \leq Q_s \leq c_s, \quad 0 \leq T_B \leq C. \quad (4)$$

Proposition 1 (accounting-cap conservation). Under A1, A3, and the specified transitions, every reachable successful-settlement state satisfies (4), irrespective of transaction ordering or whether an authorized voucher signer is honest.

Proof. Initially all settlement counters are zero. Opening a stream creates zero counters and checks allocation rule (2). Consider settlement of a voucher with cumulative amount $V > S_s$, and define $\delta = V - S_s > 0$. The contract adds δ to each subtree counter on the path from s to its root and rejects if any updated counter exceeds its cap. It similarly requires $T_B + \delta \leq C$. Counters outside that path are unchanged. If a later conversion or transfer fails, atomic rollback restores the previous state. Revocation, attestation, refund, and closure do not increase the gross settlement counters. Induction over successful transitions establishes (4). $\square$

The proposition bounds **gross settlement in accounting units**. It does not bound gas paid separately, external provider invoices, card disputes, or the USD value of a volatile token at all future prices. In a six-decimal identity-priced asset, gross voucher settlement includes both payee and protocol fee. With oracle-priced assets, accounting conservation and asset-balance conservation are distinct statements.

Let E_B be current escrow in asset base units, D_B cumulative deposits, R_B successful refund inflows, P_B settlement outflows, and L_B closure releases. Under A4 and excluding unsolicited transfers from the per-budget accounting, the corresponding flow identity is

$$E_B = D_B + R_B - P_B - L_B \geq 0. \quad (5)$$

For multiple budgets sharing one token balance, the accounting must additionally reconcile the sum of per-budget escrow records with the vault's actual token holdings. External token behavior can invalidate that reconciliation even though the accounting inequalities remain true.

Engine admission. An analogous engine invariant must count accrued usage and held reservations on every ancestor before admitting new obligations. Atomicity of the test-and-update step is essential. The contract's proof cannot be reused as a proof of pre-execution control without establishing that engine invariant and the integration boundary in A6.

5.3 The rate envelope

For an accepted escrow voucher with declared timestamp τ , the contract requires

$$V \leq r_s [\min(\tau, t_b) - t_0(s)], \quad (6)$$

when an attested breakpoint t_b exists; otherwise use τ as the endpoint. It also requires $t_0(s) \leq \tau \leq t_{\text{block}}$ and, after revocation, $\tau < \rho_s$. Without a breakpoint, the familiar engine-side form is

$$A_s(t) + H_s(t) \leq r_s \max(0, t - t_0(s)), \quad (7)$$

where A_s denotes own accrued accounting units and H_s held own reservations. The engine uses its clock, whereas escrow start time is chain-derived. Clock agreement is an operational condition, not a consequence of integer arithmetic [36].

Corollary 1 (no first-second grace). At $t = t_0(s)$ the envelope is zero. A positive cumulative charge W first fits at an integer elapsed time of at least $\lceil W/r_s \rceil$ seconds. Waiting one second suffices only when $W \leq r_s$ and the relevant chain/host clocks have advanced appropriately.

Corollary 2 (fastest-profile sizing). To admit a planned cumulative workload W_k by elapsed time $\tau_k > 0$, choose

$$r_s \geq \max_k \left\lceil \frac{W_k}{\tau_k} \right\rceil. \quad (8)$$

A bound based on average completion time can reject a faster realization. This is a consequence of the authorization model, not evidence that fast work is anomalous. Burst allowances or alternative envelopes would be distinct protocol changes.

Corollary 3 (bankable idle headroom). Because the bound is cumulative from stream opening, idle time creates later headroom, still limited by the cap. It does not bound expenditure in every sliding time window. A long-idle stream may settle a large burst. This matters both for benign bursty workloads and for the damage possible after key compromise.

Timestamp limitation. A signed signedAt field authenticates a signer’s declaration, not the actual signing time. An authorized compromised signer can attempt to backdate a voucher within the admitted window. The current checks constrain the declared-time envelope and total cap; they do not prove that all work or all signature creation occurred before revocation. Stronger temporal guarantees require a trusted measurement/timestamp mechanism, prior commitments, or a different revocation rule.

5.4 The enforcement predicate

The engine’s enforcement precedence is the first applicable reason in

$$\mathcal{E}(s, \Delta) = (\text{revoked}, \text{anomaly}, \text{sla}, \text{budget_closed}, \text{exposure}, \text{rate}, \text{payer_concurrency}, \text{cap}). \quad (9)$$

This is a priority-ordered decision procedure, not an algebraic proof that each predicate is monotone. For fixed state, configuration, clock, and observations, a deterministic implementation should return the same verdict. Across time, a rate refusal can resolve as the envelope grows; an exposure refusal can resolve after settlement; and a concurrency refusal can resolve after a reservation closes. Revocation and budget closure are terminal in the modeled lifecycle; an anomaly/SLA halt is sticky only when the appropriate persistent marker is written.

Anomaly persistence is path-dependent: a halt is terminal only when its persistent marker is written. The published HTTP SDK does not expose anomaly-envelope configuration. The runtime’s decision sequence must also be tested against the declared precedence, rather than assuming that an exported precedence constant is the executed policy. A temporary refusal therefore cannot be interpreted as a permanent stream halt.

Nor is the full list enforced by BudgetVault.settle. The contract enforces its own narrower predicate. Agreement for shared predicates is a conformance obligation; disagreement may fail safely at settlement, strand work, or create an operational incident. A rejected settlement is not evidence that the two layers never diverged.

5.5 Monotone vouchers: ordering replaces nonces

A settlement voucher has the EIP-712 typed structure

$$v_k = (\text{streamId}, U_k, V_k, \tau_k), \quad (10)$$

with cumulative meter reading U_k , cumulative accounting amount V_k , and declared time τ_k . The contract requires $V_k > S_{k-1}$ and a nondecreasing meter reading; it does not derive the amount from the meter or validate an external tariff. The engine convention sets them numerically equal, but the contract's amount authority is the signed amount field [36].

Settlement pays gross accounting delta

$$\delta_k = V_k - S_{k-1}, \quad S_k = V_k. \quad (11)$$

Proposition 2 (no duplicate gross settlement). For any accepted sequence against one stream's persistent high-water state,

$$\sum_{k=1}^n \delta_k = S_n - S_0. \quad (12)$$

Proof. Substitute $\delta_k = S_k - S_{k-1}$ and telescope. A duplicate or lower cumulative value fails the strict-advance check; a higher value pays only the increment not already settled. EVM rollback preserves the previous high-water state on failure. $\square$

The result is independent of which intermediate vouchers are omitted or presented out of order, provided the accepted cumulative amounts advance. It establishes economic at-most-once settlement, not guaranteed submission, fair payment, delivery truth, or exactly-once messaging.

EIP-712 domains bind typed signatures to a name, version, chain identifier, and verifying contract [4]. The escrow domain uses the actual chain and vault. Off-chain backends use synthetic domains; their instance isolation also depends on correct key and stream bindings. Domain separation alone is not a substitute for those bindings. Allowance credentials use separate nonces and revocation records, and the public SDK exports nonce-related functionality. Cumulative settlement therefore requires **no separate per-voucher nonce**, while credential nonces retain their own role [39].

Revocation invalidates vouchers whose declared timestamp is at or after the recorded revoke time. Earlier-dated vouchers may still settle until the budget closes. The operator's graceful sequence is to checkpoint accepted work, verify the outcome, and then revoke. This is especially important because same-second vouchers are rejected and closure provides no payee challenge window.

5.6 Idempotent metering and two-phase reservations

A metering request carries a caller-selected requestRef, unique within its stream: printable ASCII of length 1-128. Returning the prior result for a repeated reference provides retry tolerance; checking for duplicates before new admission checks lets an already billed request be acknowledged after a halt. This relies on a durable uniqueness constraint and correct record-status handling. A recovery edge case can return a stale cumulative result from a discarded record. Crash-safe retry semantics therefore require testing both uniqueness and status, not merely equality of request identifiers.

Delivery-gated application billing can use a stable delivery identifier and bill only after success. This prevents a failed delivery from being billed **only if** the adapter's success criterion is correct and no earlier irreversible charge was already made. An upstream provider may charge for unsuccessful or canceled work under its own terms. Dynamo's application receipt and the provider's invoice must not be treated as the same obligation.

For uncertain costs, the embedded engine offers reservation-style operations: reserve an upper-bound amount, execute work, then confirm within the reservation or cancel unused authority. Capacity is counted along the ancestor path when reserved; conversion to billed usage must not count the same capacity twice. However,

revocation, budget state, authorization freshness, and policy changes require an explicit confirm-time rule. Avoiding double counting does not justify bypassing every safety check.

Reservation consumption rechecks only part of the ordinary enforcement ladder. The published HTTP SDK does not expose reserve/confirm/cancel as public methods. These operations are engine mechanisms; using them for pre-execution control requires an integration that exposes them and defines confirm-time policy checks. Their existence in the engine does not imply availability through the public client.

Concurrent admission requirement. A headroom read followed by an uncoordinated provider call is insufficient: two workers can each observe the same remaining amount and both spend it. A safe implementation must atomically reserve from shared state, allocate disjoint sub-budgets/leases, or enforce an equivalent serialization rule. Offline verification alone does not solve this shared-state problem.

5.7 Fee arithmetic and the sustainability condition

Let $K = 10^4$ and let $f \in \{0, \dots, 1000\}$ be the escrow fee in basis points. For gross accounting delta δ , the contract computes

$$F_f(\delta) = \left\lfloor \frac{\delta f}{K} \right\rfloor, \quad N_f(\delta) = \delta - F_f(\delta). \quad (13)$$

For a six-decimal identity-priced asset, those values are the collector's and payee's asset amounts. For an oracle-priced asset, each is converted separately and rounded down. The fee is the rate in force **at settlement**, not necessarily the rate when the stream was opened. The administrator's ability to change it within its bound must therefore be disclosed to payers and payees [36, `_payOut`, `setFeeBps`].

Proposition 3 (zero-fee threshold and rounding loss). If $f = 0$, all protocol fees are zero. If $f > 0$, then

$$F_f(\delta) = 0 \iff \delta < \frac{K}{f} \iff \delta \leq \left\lfloor \frac{K}{f} \right\rfloor - 1. \quad (14)$$

For $n > 0$ settlements totaling $D = \sum_i \delta_i$, revenue $R = \sum_i F_f(\delta_i)$ satisfies

$$0 \leq \frac{Df}{K} - R < n, \quad R \leq \left\lfloor \frac{Df}{K} \right\rfloor. \quad (15)$$

Proof. The first equivalence is the definition of a floor equaling zero for a nonnegative argument. Apply $x - 1 < \lfloor x \rfloor \leq x$ to each fee and sum. $\square$

At the deployment default of 100 bps, a delta below 100 micro-USD produces zero **protocol** fee; gas is not thereby free. The floor loss can matter if settlement is highly fragmented, and a payee controlling checkpoint granularity may reduce fees by splitting claims. Batching, a minimum settlement size, or cumulative rather than per-delta fee assessment can change this incentive. The last alternative would be a protocol change, not a description of the contract.

Sustainability condition. Let G be the session's modeled settlement cost, converted into the same accounting denomination as revenue. The inequality

$$\sum_i \left\lfloor \frac{\delta_i f}{K} \right\rfloor \geq G \quad (16)$$

is exactly a *settlement-cost coverage condition*. It is necessary for a model in which those fees alone pay those costs, and sufficient only for that restricted model. Business sustainability additionally depends on who pays gas, operating and support costs, audit costs, losses, taxes, subsidies, and revenue retention. A more

complete contribution-margin condition is $R \geq G + O + L$, with overhead allocation O and expected loss L . A fee directed to a collector does not reimburse a gas-paying service node automatically.

The fee documentation describes a harness using recorded session shapes and PSP TEST-mode activity [44]. Such fixtures evaluate the specified cost model rather than establish profitability for every workload. On the card rail, PSP charges and a separate application fee, documented as 250 bps by default, must be modeled independently, including non-returned processing charges on refunded value. The verified-layer fee ledger, the card fee, and the vault fee are distinct mechanisms. The verified-layer fee is zero on the hosted SDK path and must not be confused with the on-chain vault fee.

Gross-to-gross reconciliation. At a completely drained, successfully reconciled billing frontier, in the identity-priced escrow mode,

$$\text{gross billed} = \text{gross engine settled} = \text{gross vault settled}. \quad (17)$$

Before that frontier, billed but unsettled claims can make the first equality false. Refunds, fees, pending claims, and asset conversion require separate columns; they must not be silently netted into settledTotal. On-chain payee receipts plus fee receipts equal the asset outflow, while oracle rounding can prevent a one-for-one comparison between asset base units and micro-USD accounting units.

5.8 Verified-value settlement

Verified-value settlement associates a stream with canonicalized service-level agreement (SLA) terms, signed observations, and a breach policy. Strict schemas admit specified integer-valued or decimal-string measurements; observation gaps and invalid signatures are recorded rather than treated as successful evidence. Consecutive breached windows can progress through breach-observed, throttle, and a persistent halt; an adequate non-breached window resets the consecutive-breach counter.

To make the policy testable, an SLA must identify the measured event, unit and tariff, observation window, minimum sample size, clock and percentile estimator, signer authority, maximum observation age, duplicate policy, and treatment of missing observations. An error-rate threshold without a denominator or a latency percentile without a window is not a complete settlement rule. The observation source may still be dishonest or collusive; a cryptographic signature does not remove that assumption.

Implementation boundary. The vault stores `slaTermsHash` but does not interpret it. It does not directly enforce a latency percentile or determine whether a video, model response, or game is correct. Anomaly configuration is not exposed by the published HTTP SDK, and higher-level runtime scheduling remains incomplete. Authenticated measurement, engine policy, and contract settlement are separate enforcement surfaces; the complete objective-fact design is not an unconditional end-to-end property of the public SDK and escrow combination [36].

Breakpoint refunds. For a stream with settled amount S_s and attested breakpoint t_b , the implemented target is

$$\Phi_s = \max\{0, S_s - r_s(t_b - t_0(s))\}, \quad \Delta\Phi_s = \Phi_s - \Phi_s^{\text{already}}. \quad (18)$$

A call executes only when the target exceeds the stored refund high-water mark. This prevents duplicate economic refunds; repeated calls may revert instead of returning a successful idempotent acknowledgment. Gross settled and allocated-cap counters are not reduced.

Proposition 4 (scope of the refund target). Equation (18) removes settlement exceeding the maximum amount authorized by the pre-breakpoint envelope. It is not generally the exact amount attributable to work delivered after the breakpoint.

Justification and counterexample. Let $r_s = 100$, $t_0 = 0$, and $t_b = 2$. Suppose no work was delivered before t_b , and 100 units were settled later before the breakpoint was attested. Equation (18) gives $\Phi_s = \max(0, 100 - 200) = 0$, even though all 100 delivered units were post-breakpoint. A rate envelope is an upper bound, not a record of when work occurred. Recovering the exact post-breakpoint entitlement requires authenticated cumulative earned value at the breakpoint, or equivalent per-unit evidence, rather than this envelope alone. $\square$

Funding of refunds: implemented versus proposed. In the vault, successful settlements transfer funds immediately. `refundAfterBreakpoint` pulls the refund from the service node with `ERC-20 transferFrom`; it requires the payee's approval and sufficient transferable balance. The refund is gross, including the earlier protocol-fee component, while the fee collector is not debited. It is therefore neither a solvency-independent refund nor a refund automatically paid from a funded holdback reserve [36, `refundAfterBreakpoint`].

A proposed **buyer-whole-first** mechanism would retain eligible contingent value in enforceable escrow or collateral until an agreed dispute window ends, then pay eligible refunds from that reserve before distributions. A bookkeeping finality delay alone cannot supply the reserve. Its amount, control, priority, liquidation or transfer rules, and exhaustion behavior require explicit specification and testing. After irrevocable release, recourse is limited to remaining collateral and applicable counterparty obligations. This reserve-backed mechanism is distinct from the implemented payee-funded refund path.

5.9 Oracle-priced assets and the round-trip lemma

Let a be token base units, d the **token's** decimals, $p > 0$ a price normalized to micro-USD accounting units per whole token, $K = 10^4$, and $k = K - h_o$, where haircut $0 \leq h_o < K$. The registry implements

$$T_p(a) = \left\lfloor \frac{k}{K} \left\lfloor \frac{ap}{10^d} \right\rfloor \right\rfloor, \quad A_p(x) = \left\lfloor \frac{x 10^d K}{pk} \right\rfloor. \quad (19)$$

The inverse divides by the haircut-adjusted price. It is not a second haircut multiplication. Feed decimals are normalized by the adapter; confusing them with token decimals changes the economic scale [37].

Proposition 5 (fixed-quote round-trip non-inflation). For one fixed valid price, haircut, and decimal configuration, and arithmetic within implementation bounds,

$$A_p(T_p(a)) \leq a. \quad (20)$$

Proof. Since $T_p(a) \leq apk/(10^d K)$, multiplying by the positive factor $10^d K/(pk)$ yields at most a . Flooring that quantity cannot increase it. $\square$

The result does not require each conversion to be individually non-expanding: the two conversions use different units and one factor can exceed one. It also does not hold across arbitrary quote changes. If the price halves between conversion into accounting units and conversion back, the same accounting claim can correspond to approximately twice as many tokens. A haircut is a collateral buffer, not a proof against unlimited depreciation or oracle manipulation.

For payout splitting at one fixed quote, $A_p(x) + A_p(y) \leq A_p(x + y)$; the difference is less than two token base units and, for two terms, at most one. Remaining dust stays in the vault accounting until release. Asset

decimals, checked multiplication ranges, minimum payable amounts, and rounding residues belong in conformance tests.

Fail-closed pricing. Registry conversions reject unlisted assets, zero prices, zero/future timestamps, and stale observations. The separate adapter applies a deviation band and an observation baseline, with a permissionless baseline-update operation. This is a bounded-change check, not a defense against all economically harmful drift [37]. `closeBudget` does not consult the oracle, so stale prices alone do not block its conversion-free release path. Closure still requires all streams revoked, sufficient gas, functioning token transfers, and an executable chain.

Stable listings are exactly six-decimal and identity-priced. That is a declared peg assumption; the registry does not test real-world peg stability. Add-only asset/oracle configuration avoids retroactive administrator edits but also prevents correction of a bad listing in that deployed registry.

5.10 Card-rail epoch settlement

The card backend uses a manual-capture authorization rather than a stored protocol balance. Funding rounds an accounting cap upward into cents:

$$A_{\text{cents}} = \lceil C/10^4 \rceil. \quad (21)$$

The cap on accepted accounting usage remains C ; the rounded authorization need not be fully captured. Capture strategy is capability-dependent: `epochCapture` requires multicapture support, `finalCapture` permits one final capture, and `auto` selects between them. The implementation is restricted to **Stripe TEST mode**; a tested code path is not a production payment-service approval [30].

Let chargeable accounting amount be $Q = \sum_s (S_s - R_s^{\text{accounting}})$. With gross captures X and PSP refunds Y , both in cents, the documented capture-debt model is

$$Z = \lfloor Q/10^4 \rfloor, \quad X_{\text{net}} = X - Y, \quad \Delta_{\text{cap}} = Z - X_{\text{net}}. \quad (22)$$

A positive debt can be captured only when the strategy and remaining authorization permit it. A nonpositive debt is not a negative capture; it belongs to the refund/reconciliation path. The governing safety constraint is

$$\sum \text{gross captures on an authorization} \leq A_{\text{cents}}. \quad (23)$$

Refunded value does not restore the same authorization's capture capacity in this model. Re-earned value after a refund can therefore require a new authorized ceiling. The backend's renewal decline is a state result (renewed: false, with decline detail) that marks the ceiling dead; it must not be ignored by callers that handle only exceptions.

Payout targets floor to cents. Sub-cent residues are recorded at release, but recording a liability or residue is not equivalent to paying it. The residual ownership, eventual aggregation, and disclosure policy must be explicit. Card reconciliation can tolerate a sub-cent shortfall under the specified model, while over-capture is a failure; this is a rail-specific tolerance, not identity with escrow arithmetic.

Authorization expiry, multiple-capture eligibility, transfer reversal, chargebacks, and PSP outages constrain liveness. The backend's modeled seven-day authorization TTL is not a universal rule about every card authorization. A production adapter must use the actual supported authorization and capture rules and distinguish API acceptance, capture, transfer, payout, and dispute finality [30].

5.11 Subscriptions as recurrence

The subscription design is a recurrence wrapper: a fresh period creates a new funded budget from a declared template. On cards, each period requires an authorization rather than interpreting card-on-file possession as unlimited spending authority. The template specifies whether unused ceiling is released or contributes to the next period's cap. Rollover of a *limit* must not be confused with automatic transfer of money or revival of an expired authorization.

Renewal failure stops new unfunded accrual; previously funded obligations retain their separately defined settlement rights. Period-open, renewal-decline, and close events provide a versioned integration interface. Consumers still need durable deduplication and a recovery/query mechanism; webhooks alone do not guarantee delivery.

Subscriptions and some batch operations are engine capabilities rather than fully exposed public HTTP-client methods [42]. Production subscription support requires a defined wire interface, an end-to-end authorization lifecycle, and consent and compliance review. The recurrence model does not by itself establish those operational capabilities.

5.12 Exposure ladders and the timing trade-off

The engine model caps unsettled service-node exposure at a configurable $E_{\max}$, defaulting to USD 1,000 in accounting units. An exposure bound must identify its scope—instance, budget, service node, or combination—and include pending reservations. A bound of $E_{\max}$ plus one in-flight request applies only to serialized admission with one such request; with multiple unreserved concurrent requests, the additional liability is their sum. These conditions determine the actual bound on provider credit.

The payer-side degradation ladder uses remaining budget headroom H_B and

$$q = \lfloor 100 \max(0, H_B)/C \rfloor. \quad (24)$$

The reported defaults are: at least 50% gives full; 25-49% gives throttled; 10-24% gives checkpoint-required; below 10% gives refuse. Throttling constrains request size, and checkpoint-required blocks additional accrual until outstanding claims settle. These are conservative policy thresholds, not unavoidable consequences of a funded cap. In particular, a deliberate refusal may leave budget unused.

Checkpoint cadence. Let $c > 0$ be an interval, $g > 0$ cost per checkpoint, $r > 0$ a constant admitted accrual rate, and $\lambda > 0$ a risk-price coefficient with units chosen so that both terms below are cost per unit time. A peak-exposure approximation gives

$$J(c) = \frac{g}{c} + \lambda r c, \quad c^* = \sqrt{\frac{g}{\lambda r}}, \quad J(c^*) = 2\sqrt{g\lambda r}. \quad (25)$$

Derivation. $J'(c) = -g/c^2 + \lambda r$ vanishes at c^* ; $J''(c) = 2g/c^3 > 0$, so this is the unique unconstrained minimum. If the risk term instead prices the time-average exposure of linear accrual, the average is $rc/2$ and the optimum is $\sqrt{2g/(\lambda r)}$. Choosing between them is a modeling decision.

A deployment must also impose feasible bounds: maximum allowed latency, expiration, risk ceilings, block inclusion variability, and batching rules. The constrained optimum is the unconstrained value clipped to a feasible interval when that interval is nonempty. At zero risk price or zero accrual, no finite interior optimum follows from (25); at zero checkpoint cost the model favors the smallest feasible interval. Bursty workload and variable gas costs require a different objective or sensitivity analysis.

Holdback. A proposed holdback h_b makes claims conditionally final until a declared release criterion, but the number of checkpoints within a trailing window depends on phase and can differ by one from $\lfloor h_b/c \rfloor$. More importantly, a time label does not imply refund-reachable funds. The vault pays immediately, and the hosted runtime does not schedule the finality polling required by a nonzero holdback configuration. Setting `holdbackSeconds: 0` addresses that scheduling limitation; it does not make chain finality immediate or create a solvency-independent refund reserve [45].

6. Security Properties and Analysis

6.1 Scoped credentials and bounded blast radius

A correctly provisioned agent receives narrowly scoped authority rather than the budget owner’s unrestricted key. For a voucher signer authorized on only one stream, the contract limits gross accepted settlement to the minimum remaining headroom across that stream’s ancestor path and its budget. The rate envelope adds a declared-time constraint. Foreign-stream rejection additionally depends on unique or correctly bound session keys; reusing the same authorized key across streams changes the compromise scope [36].

This bound does not apply unchanged to a leaked owner API token, budget-owner key, treasury key, or PSP credential. The vault accepts its budget owner as a fallback voucher signer across every stream in that budget. The hosted core also handles signing material and can be a substantial trust boundary. Local compromise may affect confidentiality and future authorized expenditure even when cap conservation survives.

Allowance-credential revocation requires the enforcing gateway to learn the relevant revoked nonce or updated policy. An offline verifier with a stale revocation view cannot know a remote revocation instantaneously. Short lifetimes, bounded refresh intervals, and disjoint spending leases can bound exposure; their parameters must appear in the security claim.

OWASP’s taxonomy is useful for organizing threats, not certifying conformance. ASI01 concerns agent goal hijack, ASI02 tool misuse and exploitation, and ASI03 identity and privilege abuse [16]. Caps can reduce the financial consequences of compromised goals and tool use; they do not prevent prompt injection, make tools safe, or eliminate identity abuse.

6.2 Threat classes and their answers

Table 2. Defenses and residual conditions.

Threat	Supported defense	Residual condition or required evidence
Leaked scoped key	Stream/ancestor/budget accounting caps; signer checks	Key isolation, timestamp trust, and non-bypassable provider access
Replayed voucher	Persistent cumulative high-water state	Correct domains, instance bindings, canonical-chain observation
Stale or manipulated price	Freshness and deviation checks; payout rejects if underfunded	Feed trust, baseline drift, collateral loss, depeg and token-issuer powers
Runaway loop	Cap, cumulative envelope, configured admission policy	Idle headroom, concurrent work, external costs; SDK anomaly feature unavailable as documented
Crash during settlement	Durable local records and signed frontier	Cross-system recovery windows remain; reconcile actual chain/PSP outcomes

Threat	Supported defense	Residual condition or required evidence
Revocation race	Reject declared timestamps at/after revoke	Same-second loss, backdating, delayed inclusion, gas limits, early close
Malicious administrator	No arbitrary vault-principal withdrawal; bounded fees	Owner/collector changes, fee timing, different roles' keys, token controls
Counterparty dispute	Signed and digest-checked records	Trusted identity anchors, truthful observations, completeness, applicable procedure

Revocation liveness. The implementation recursively writes every node in the revoked subtree. Depth is limited to eight, but breadth is not comparably bounded. A sufficiently large subtree can exceed a transaction's gas budget, so "one-call instant subtree revocation" requires a tested size limit and successful inclusion. Revoking smaller subtrees is a mitigation, not an unconditional atomic revocation guarantee for an arbitrary network [36, `_revokeSubtree`].

Payee liveness. After all streams are revoked, the owner may immediately close a budget. Unsubmitted otherwise valid vouchers then become unseizable. This is an explicit counterparty risk. A guaranteed claim-submission window would require a different closing protocol. Finite checkpoint cadence reduces exposure but does not eliminate the race.

6.3 Fail closed as the system property

Malformed signatures, nonadvancing cumulative claims, and invalid conversion inputs cause relevant contract transitions to revert. That is a meaningful financial safety boundary. However, fail-closed operation can preserve principal while denying service or withholding legitimate payment. It cannot simultaneously guarantee instantaneous revocation, uninterrupted offline service, and globally shared uncoordinated spending authority.

A failure taxonomy should distinguish **rejection before an obligation**, **rejection after external work**, **unknown backend outcome**, **confirmed transfer with missing local acknowledgment**, and **irrecoverable application state**. Only the first directly supports a "no new cost" claim. Unknown outcomes must quarantine the affected authority until reconciliation; treating a timeout as proof of no transfer can double-pay or double-allocate.

The administrator surface is limited in the contract, but absence of an arbitrary withdrawal function is not administrator powerlessness: fee changes alter payee proceeds. Testing intended negative capabilities is useful evidence; it is not a proof about arbitrary future upgrades or every external dependency [36].

6.4 Evidence and dispute verification

Dynamo's evidence design assembles session records, vouchers, credentials, attestations, settlement references, refund records, and a content-digest manifest in an evidence/1.0.0 bundle. An offline verifier can check included signatures, recompute digests, and recompute accounting totals. The evidence documentation reports a 71-check offline demonstration and describes operator-mediated export rather than a supported public SDK export method [43]. The objective is dispute-oriented evidence with explicit verification assumptions.

Three boundaries are essential. First, verification material included in a bundle is not automatically trusted: replacing both a public key and all signed records can produce a self-consistent counterfeit unless an identity or chain root is independently pinned. Second, signature validity does not prove measurement truth, the

completeness of the event set, or current chain/PSP finality. Third, an offline snapshot cannot learn subsequent revocation or a later reorganization without new authenticated information.

The per-call SDK receipt is a separate artifact: `dynamo-receipt-v1` is signed with EIP-191 over the serialized payload by a session key. Settlement vouchers use EIP-712, while allowance and fact credentials use JWS/ES256K. These formats have different purposes and verification rules. A per-call receipt is not a complete evidence export.

A reproducible evidence fixture needs a pinned trust root, canonicalization rules, expected success and failure outputs, a completeness boundary, and offline verification commands. A one-byte tamper test evaluates the authenticated envelope that contains that byte; a change outside the envelope need not have the same effect. Results must identify the covered bytes and checks. Compatibility with Visa TAP, Mastercard Verifiable Intent, or a card dispute process requires separate conformance evidence [28, 29].

6.5 Mechanism-level honesty

One-way cap allocation and **same-second clipping** are explicit conservative choices. The former trades allocation flexibility for simpler authority accounting. The latter declines ambiguous declared-time claims at the revocation boundary. Neither replaces a complete threat model: reserved caps can become unusable, and valid pre-revocation work can go unpaid if settlement is delayed.

The graceful public-client sequence is `checkpoint({force: true})`, `verify completion`, then `revoke()`. `force` bypasses the checkpoint-due policy, not signature, funding, monotonicity, or finality checks. If the backend revocation fails after local commitment, local state is not proof of chain success; the hosted retry path does not automatically re-drive that backend operation. The affected authority must remain stopped until its backend state is reconciled.

Mechanism-level honesty also requires identifying simulations, TEST-mode PSP calls, testnet value, and production transfers distinctly. A signed receipt and a realistic transaction hash-shaped string are not interchangeable. An unavailable reconciliation result is **unverified**, not automatically reconciled and not necessarily a mismatch. These distinctions are part of the protocol's auditability rather than cosmetic qualifications.

7. Implementation and Evaluation

7.1 The build

Dynamo comprises a public escrow contract family, open client packages, gateway tooling, and conformance artifacts. The stream engine and several merchant and evidence components are proprietary [36-39, 48]. Public client code makes the interface inspectable without making the full runtime independently reproducible. Feature availability follows the component and integration boundaries described below.

The contract suite includes BudgetVault, an add-only AssetRegistry, and a Chainlink-shaped price adapter. The public build configuration pins Solidity 0.8.24 with optimization enabled at 200 runs. The public SDK is `@dynamoprotocol/sdk 2.1.0`, used with owner API 1.1.0 [38, 39, 42]. Integrations must check the configured instance's reported interface and funding modes, rather than infer them from a package name alone.

The public applications illustrate two service workflows. **DreamCade** turns keywords into a browser game with a capped repair loop. **DreamReel** turns keywords into a video series. Dynamo's public site reports both using the escrow rail on Base Sepolia with testnet stablecoins and reports 43/43 end-to-end checks for DreamReel [40]. These are project-reported testnet deployments, not evidence of production-money readiness or independent replication.

7.2 The package map

Table 3. Components, access, and implementation scope.

Package or component	Stated license/access	Role and limitation
contracts	BUSL-1.1, public source	Vault, registry, price adapter; directly inspectable contract semantics
engine	Proprietary	Budgeting, metering, backends, higher-level policies; runtime behavior is project-described
funding-card	Proprietary	Card backend; documented implementation restricts keys to TEST mode
@dynamoprotocol/sdk	Apache-2.0	Published thin HTTP client, v2.1.0; does not embed the core
plugin-litellm	Apache-2.0	Python gateway integration, published as litellm-dynamo
gateway-kong, gateway-apisix	Apache-2.0	Lua integrations; a shared vendored decision core reduces implementation independence
mcp	Apache-2.0	MCP monetization adapter
dynamo-guard	Apache-2.0	Capped proxy for compatible clients
conformance-*	Apache-2.0	Public protocol-specific checks and test vectors
events, merchant-api	Proprietary	Webhook outbox and payee-facing verification interfaces
evidence, console	Proprietary	Evidence assembly and read models; operator-mediated export; no public-client export method

The licensing boundary is component-specific [48]. Publicly inspectable source is not necessarily permissively licensed: BUSL-1.1 has use restrictions before its change terms apply. Runtime exposure is also separate from package existence; not every engine capability is available through the published client.

7.3 Test surface

Reported test coverage comprises approximately 490 TypeScript tests across 16 packages, 170 Solidity tests, 34 Python tests, 259 Lua assertions per pinned gateway image, at least 22 property suites with 10,000 randomized runs each, and a 10,000-vector decision comparison across four plugin implementations in three languages. These are project-reported inventory and campaign figures. They describe the intended evaluation surface, not an independent certification or a proof of complete state-space coverage.

The public Foundry configuration specifies **10 invariant functions x 2,100 runs x depth 100 = 2,100,000 configured handler-call opportunities**, with 14 handler selectors in the vault harness. Configuration is not an execution log; early termination, reverts, framework behavior, and campaign structure can change meaningful transition coverage. In particular, `fail_on_revert = false` requires successful and rejected calls to be reported separately [38].

Table 4. Contract invariant families and their scope.

ID	Property family	Required qualification
I1	Network/subtree caps and accounting conservation	Gross counters; modeled transfer semantics; Proposition 1
I2	Rate-envelope conformance	Declared timestamps and configured clocks, not measured instantaneous burn
I3	Refund bound/idempotence	Envelope excess and available payee funds, not full recovery of every failed delivery
I4	Revocation cutoff	Rejection of timestamps at/after revoke, not all post-revoke submissions
I5	Subtree revocation ordering	Successful transaction and executable subtree size
I6	Replay rejection	Persistent stream high-water state and correct domain/bindings
I7	Scoped-key confinement	No cross-stream key reuse or owner-key compromise
I8	Oracle fail-closed checks	Defined invalid inputs, not universal price correctness
I9	Exact asset outflow conservation	Include both payee and collector transfers and asset rounding
I10	Batch/sequential equivalence	Equivalent initial states and successful atomic execution
I11	Administrator restrictions	Bounded fee/collector powers remain; not literally no power

Property-based testing searches sampled execution paths; it does not prove universal correctness. The mathematical propositions in Section 5 are conditional arguments about the stated transition system, not machine-checked proofs of the compiled implementation. Engine unit and property tests using in-memory databases also do not establish file-backed WAL durability under arbitrary crashes. Crash testing requires the actual persistent-storage configuration.

Reproducibility manifest. A reproducible result identifies repository and commit, dependency lockfile, compiler and runtime, operating system, chain and deployment address where relevant, seed, test distribution, trial count, successful and rejected operations, invariant definitions, full logs, and artifact hashes. Cross-language agreement also requires an independent reference specification and boundary and adversarial vectors. Agreement among copies of the same logic is useful but is not independent algorithmic validation. Appendix C defines the evidence boundary.

7.4 The integrator contract, the sustainability harness, and the conformance methodology

Integrations that touch settlement are evaluated against nine acceptance requirements [46].

Requirement	Acceptance criterion
R1: Event idempotence	Deduplicate a stable authenticated event identifier durably, including crash/retry cases
R2: Credit only on finality	Credit only the particular verified final frontier; do not infer all newer claims are final from an older final row
R3: Duplicate/reorder tolerance	Apply cumulative advances without double counting; quarantine unknown backend outcomes
R4: Conformance	Name the protocol version, scheme, target implementation, and covered checks

Requirement	Acceptance criterion
R5: Payee-side verification	Verify authority, amount, domain, and funding independently of a caller's unsupported assertion
R6: Per-mode onboarding	Supply the required chain/asset or PSP configuration and explicitly labeled sandbox credentials
R7: Monetary units	Keep physical service units, micro-USD accounting amounts, token base units, and card cents distinct
R8: Version discipline	Pin compatible interfaces and document additive changes and deprecations
R9: Sandbox/production parity	Reuse logical interfaces while separately testing real funding, transfer, recovery, and finality conditions

R7 requires dimensional correctness: a physical service measurement, a priced micro-USD accounting amount, an ERC-20 base-unit amount, and a card-cent amount are separate quantities even when a particular conversion is an identity. R9 requires both logical interface conformance and backend-specific testing; simulation cannot reproduce every production authorization, transfer, or recovery condition.

The integration sequence is: authenticate the request; reserve bounded authority where needed; execute; determine billable delivery under the agreed policy; meter idempotently; checkpoint; reconcile independently; and retain the evidence. A gateway may perform some operations asynchronously, but asynchronous reporting cannot substitute for a correct pre-execution spending boundary. The plugin wire records and signs without checkpointing, so a host must drive settlement explicitly.

Fee harness. Each fixture's gas and PSP costs must be recomputed in consistent units, with the cost bearer identified and unsuccessful operations, refunds, and nonreturned charges included. Results should identify which fixtures satisfy (16), together with sensitivity to fee rate, gas price, payment size, refund fraction, and checkpoint count. The published methodology includes TEST-mode PSP fixtures; those fixtures evaluate payment arithmetic and integration, not production-money economics [44].

Conformance methodology. The public kit describes twelve checks covering challenge shape, supported schemes, denial ordering, and merchant protection. Passing establishes only the defined coverage. Reproducing the reported 10,000-vector comparison requires the generator, seed and distribution, target binaries, and authenticated results. The two Lua gateways share a byte-identical decision core: their agreement tests integration consistency but is not evidence from two independent algorithms. Compatibility claims must name Dynamo's dynamo-allowance surface and the relevant x402 version, scheme, and extensions [42, 48; 23-26].

Economic illustration. A USD 0.50 job with a 1% protocol fee and assumed USD 0.0016 gas costs USD 0.0066, or **1.32%**. At zero protocol fee, the same assumed gas cost is **0.32%**. A hypothetical card tariff of 2.9% plus USD 0.30 costs USD 0.3145, or **62.9%**, for one separate USD 0.50 charge. These are arithmetic consequences of specified assumptions, not a current universal tariff, a measured population average, or a comparison against aggregated card billing. Funding, bridging, conversion, failed calls, refunds, taxes, and amortization can change the result.

Comparative evaluation. A controlled evaluation should compare the same service traces under Dynamo, a transactional gateway budget, current x402 methods, an MPP session, and aggregated card billing. Report cap violations, legitimate-work refusal rate, uncollectible delivered work, recovery success, p50/p95/p99 admission and settlement latency, total lifecycle cost, and evidence completeness. Vary concurrency, subtree

width/depth, request size, price uncertainty, outage duration, key compromise, and dishonest observations. Demonstrations without those controls establish feasibility, not relative superiority.

7.5 Demonstrated behaviors

The reported demonstrations exercise selected failure and settlement paths. Each result is bounded by its configuration, input trace, and execution environment; none alone establishes a universal guarantee.

Reported demonstration	Evidence-supported interpretation	Present limitation
Recreated USD 47,000 runaway stopped at USD 63	A specified guarded trace stopped earlier than its matched baseline	Baseline is a recreation/counterfactual, not independently verified avoided loss; published SDK does not expose anomaly configuration
kill -9 billing at the last signed voucher	Crash recovery for the tested injection point and storage setup	Does not cover every chain/PSP/local commit boundary or budget rehydration
One authorization, two payees, cent reconciliation	Capture, split, refund, reversal arithmetic for that processor session	Distinguish PSP TEST mode from production charges and name capture capabilities
Renewal decline freezes dependent streams	No admitted new usage after the tested unfunded renewal	Requires state-result handling and timing/concurrency coverage
Stale/deviant price rejection	Rejection of the tested stale/out-of-band observations	Does not prove all oracle manipulation is prevented or all collateral remains redeemable
71 offline evidence checks; one-byte change produces three failures	Verification of the exact bundle, trust root, and tamper fixture	Does not by itself prove provenance, completeness, legal acceptance, or current finality

Each demonstration requires an executable script, input trace and configuration, pinned dependencies, expected result, full output, and a reproducible linkage to the tested build. Testnet and production claims additionally require actual chain or PSP references and confirmation rules. For an application workflow, the evidence includes payment and refund explorer records, a verbatim signed receipt, and the aggregate reconciliation result. A product page or pass count cannot substitute for this replication package.

7.6 Deployment shapes

Deployment options are a self-hosted licensed core and dedicated hosted instances. Multi-tenant operation is a future architecture rather than an implemented deployment mode. The public site reports Control Mode and Base Sepolia escrow use; production card and mainnet activation remain per-instance and review-gated [40]. Activation controls must be bound to the actual runtime image, configuration, and environment.

Dedicated hosting isolates instances but does not automatically remove operator access to signing material. Self-hosting shifts operational responsibility to the user. Multi-tenant support would require tenant-scoped keys, authorization, durable budgets, resource isolation, and demonstrated cross-tenant failure containment; it is not a labeling change.

Performance evaluation distinguishes local credential verification, an HTTP metering call, voucher creation, checkpoint inclusion, and finality. Offline credential verification can remove a remote authorization round trip; it still incurs local processing and does not establish zero overhead for the complete request path.

Simulated application measurements, isolated timings, and testnet results must not be presented as production latency distributions.

8. The Grand Vision: From Protocol to the Finance and Economic Layer of the AI Agent Economy

This section and Section 9 describe a forward-looking design space. They do not establish deployment, adoption, revenue, or token issuance. Their prerequisites are distinct from the settlement invariants in Section 5.

8.1 The thesis

Dynamo’s long-horizon thesis is that increasingly autonomous workflows will require a common economic control surface: authority to spend, limits on delegated obligations, records of performance and payment, and procedures for reconciliation and disputes. The thesis identifies private data and workflows, provider neutrality, and economical granular settlement as potential sources of durable value as model capabilities improve. These are strategic hypotheses to test against actual adoption and willingness to pay, rather than deductions from model capability trends.

The proposed finance and economic layer groups budgeting, bookkeeping, escrow, refunds, evidence, exposure limits, and audit support around a shared object graph. A budget tree represents delegated authority; cumulative vouchers represent payment claims; backend records represent transfers; and evidence packages connect those claims to their asserted basis. Such integration can reduce inconsistent records between components. It does not eliminate the need for measurement, operational controls, accounting policy, legal agreements, or independent assurance.

The ambition is therefore institutional as well as technical, but the scope remains precise: Dynamo supplies primitives that can support economic institutions, not a claim that a voucher substitutes for every function of an accountant, auditor, creditor, or court. The currently deployed spend-control surface is an initial application of that architecture, not empirical validation of the entire thesis.

8.2 The economic-layer stack

The following correspondence describes the economic functions supported by the architecture and the limits of each analogy.

Institution	Conventional form	Dynamo primitive and scope
Budgeting	Plans and departmental limits	A funded budget and delegated caps; settlement conservation does not by itself bound off-rail liabilities
Bookkeeping	Ledgers and reconciliation	Cumulative vouchers and transfer records; complete accounting also requires costs, liabilities, refunds, and external records
Escrow	Conditional release by a trusted or programmed intermediary	The vault holds budget assets; general delivery-conditioned holdback remains a stronger design than the current payout path
Net terms	Deferred invoices and negotiated payment dates	Checkpoint and holdback policies govern timing; a refund window is not equivalent to underwriting 30/60/90-day credit
Audit	Independent examination and assurance	Portable evidence supports verification; it is not an independent audit opinion or proof that the record is complete

Institution	Conventional form	Dynamo primitive and scope
Credit limits	Underwritten counterparty exposure	Explicit limits on unsettled work; these bound modeled exposure but do not price default risk or guarantee recovery
Tax base	Public transaction taxation	An explicit protocol-fee row; this is a private service charge, not a tax or fiscal authority
Reputation	Credit histories and references	Authenticated settlement history as an input; wash activity, collusion, and identity changes still require controls

The individual primitives have substantial prior art. Dynamo’s contribution is their proposed composition with a bounded delegation tree and a common funding interface. The relevant validation question is whether that composition reduces reconciliation failures and implementation cost without creating unacceptable trust, latency, or availability burdens. Common object shapes are useful, but do not make card, ledger, and escrow semantics identical.

8.3 The arc

Phase I - the wedge. The initial application is spend control for coding-agent users and AI-native teams. A unilateral control tool can be useful even before an external seller adopts the protocol. The reported USD 47,000-to-USD 63 recreation illustrates the intended failure intervention; it is not a measured estimate of typical savings or a substitute for a reproducible matched baseline (Section 7.5).

Phase II - the sell side. The same abstractions can support per-use monetization of tools, APIs, model endpoints, data, and human services. A founding-seller cohort with an introductory fee waiver is a possible adoption mechanism. The illustrative escrow and card application fees are 1% and 2.5%, respectively, with rail-specific costs additional. A comparison with hypothetical marketplace take rates of 20-25% is not like-for-like: marketplaces may bundle acquisition, support, insurance, disputes, or other services absent from a settlement fee. Evaluation must identify the service bundle, date, jurisdiction, payment size, and total lifecycle cost.

Phase III - the market. Multi-vendor workflows could allocate a parent budget across sellers and subcontractors, allowing agents to hire agents or humans under a common accounting ceiling. A payment integration is not yet a marketplace: discovery, identity, contract formation, cancellations, service measurement, and dispute resolution remain necessary. Market formation should be evaluated through completed cross-party workflows, independent active counterparties, repeat use, reconciliation quality, and sustainable unit economics rather than transaction volume alone.

Phase IV - the economic layer. Settlement history could inform portable reputation, disputes, fleet treasury policy, and exposure management across compute, data, bandwidth, energy, and human work. It complements identity and validation systems such as ERC-8004 rather than replacing them [13]. Payment history is informative only to the extent that identities, independence of counterparties, and the underlying economic substance can be assessed. The AI agent economy is the proposed entry market, not an architectural requirement that every participant be an AI system.

8.4 The adoption ladder

The intended ladder proceeds from Control Mode, through testnet and processor-sandbox integrations, to reviewed money-rail deployments. Reusing budget and stream concepts can reduce migration effort, but each step must exercise its actual backend failure modes. A no-money ledger cannot establish that a real authorization survives expiry, that an ERC-20 remains transferable, or that a refund can be recovered.

The adoption strategy assigns Control Mode to acquisition, card payments to a potential revenue channel, and the escrow/token ecosystem to reach and interoperability. These are proposed business roles, not measured channel performance. The distinction between an existing stablecoin settlement rail and a future DYNAMO utility token must also remain explicit: use of the escrow backend does not require the proposed token.

DreamCade and DreamReel provide project-reported examples of Base Sepolia application integration [40]. Progress from testnet to production depends on backend-specific acceptance evidence, external review, and operational readiness, not solely on changing a funding selector.

8.5 Why a dedicated ledger eventually

A dedicated ledger is one possible response to three pressures: shared-chain transaction cost and throughput, governance neutrality among competing applications, and funding the security and operation of the settlement infrastructure. None makes a new chain inevitable. Batching, improved checkpoint policy, existing rollups, or other shared execution environments may be preferable at a given workload and risk level.

A decision to build a dedicated ledger should compare observed settlement demand and service-level requirements against alternatives, including execution fees, data availability, validator incentives, bridge exposure, censorship resistance, recovery, and integration cost. Host-chain fees purchase security and availability; they are not simply lost value. A native gas asset changes the security budget and stakeholder incentives, but does not automatically improve them.

The neutrality argument. An infrastructure provider that also competes with its users can create concerns about pricing, access, and governance. Neutrality can be supported by published interfaces, bounded administrative powers, self-hosting, transparent fees, and credible change procedures. Those features are assessable commitments, not proof that conflicts of interest disappear. Conversely, private ownership alone does not establish that neutral operation is impossible. The analogy to membership-governed payment networks is an institutional design analogy, not a demonstrated causal argument for token governance.

Hyperliquid provides a non-identical example of a product-linked ecosystem with a native gas asset and protocol-related token purchases. Its documentation distinguishes fee recipients and describes the assistance fund and HyperEVM gas mechanics [31, 32]. Those mechanisms inform design comparison, not a gross-fee allocation or valuation rule transferable to Dynamo. The proposed sequence is to demonstrate useful, financially supportable activity before introducing an asset or a new ledger. No chain-launch threshold or implementation is specified here.

8.6 The interplanetary extension

The long-horizon ambition is an **Interplanetary AI Economic and Finance Layer** [40]. Potential workloads include remote inference, relay bandwidth, observation data, compute time, and scheduled power availability. Payload-neutral accounting and portable signed evidence are relevant primitives. Operation across intermittent or long-delay links nevertheless requires additional authority-allocation, timing, and settlement mechanisms; it is not an established capability of the present implementation.

Delay-tolerant networking explicitly accommodates intermittent connectivity and store-and-forward delivery [35]. A disconnected verifier may check signatures and digests against previously trusted keys, but cannot thereby learn a newly issued revocation, current collateral balance, or another partition's latest spend. An oracle freshness rule calibrated for terrestrial connectivity may halt otherwise legitimate remote operations. Similarly, terrestrial consensus and confirmation assumptions cannot simply be carried across large propagation delays.

A possible research design preallocates disjoint local authorities $C_1, \dots, C_m$ satisfying

$$\sum_{j=1}^m C_j \leq C. \quad (26)$$

Each domain enforces its allocation without consulting the others; reconciliation occurs when communication resumes. Maintaining safety requires non-overlapping authority, domain-bound vouchers and keys, and a rule preventing released or expired authority from being reused while a disconnected domain can still spend it. Expiry and revocation must account for bounded clocks or an explicitly asynchronous protocol. Such a design trades liquidity and utilization for partition safety; it is not a claim of instantaneous global revocation.

Other research requirements include measurement integrity on remote hardware, settlement finality across domains, local asset liquidity, attestation availability, fault recovery, and adversarial simulations of long disconnections. Offline evidence verification is a useful component of this agenda, not an offline consensus protocol. The execution order remains **Earth first**: establish terrestrial safety, operability, and demand before presenting extraterrestrial use as a demonstrated capability.

9. The DYNAMO Token (Design Space)

The DYNAMO token is a conditional design proposal. This section does not constitute a token offer, issuance schedule, allocation entitlement, or promise of returns. Issuance and deployment would require a complete mechanism specification, readiness assessment, and appropriate legal review. Numerical parameters below are candidate policy inputs, not empirically validated launch criteria.

9.1 Design principles

1. Revenue first, token later. Candidate readiness thresholds are at least USD 250,000 in monthly recurring revenue, with USD 500,000 representing a stronger scenario; a recurring share of at least 70% of total recognized revenue; at least three stable months; and approximately 10,000-20,000 active participants. Monthly recurring revenue must exclude nonrecurring and self-generated activity. Community size requires a defined activity window and resistance to duplicate identities. These thresholds are proposed readiness policies, not validated predictors of a successful or safe launch. Usage records may support eligibility analysis subject to consent, data quality, and anti-gaming controls.

2. Community-earned distribution. The proposed policy excludes venture allocations, insider funding rounds, and a development-team share of ongoing protocol fees. It envisages seasonal usage-based eligibility, anti-gaming rules, automatic distribution rather than a claim flow, and at least 70% allocated to the community. A team allocation would have a minimum one-year cliff followed by linear vesting. The community minimum leaves no more than 30% for all other allocations combined. These are candidate design policies, not existing rights. Automatic distribution still requires rules for eligible recipients, consent, mistaken transfers, jurisdictional restrictions, and disputes.

3. Mechanical, fee-funded buybacks. A candidate buyback share of approximately 97% is represented by the scenario parameter $\rho = 0.97$, not a live protocol rule or an established optimum. The policy must define whether its base is gross fees or distributable net revenue. If applied to gross fees, only 3% remains for costs and reserves not funded elsewhere. If applied after costs, the smaller distributable base must be explicit. A programmed rule can constrain discretion but still needs liquidity, execution, pause, custody, and failure policies. It does not guarantee a market price or token-holder return.

4. Coverage and valuation discipline. A target of at least 50% of modeled unlock-and-sell pressure and an illustrative 8.5-times-annual-buyback multiple are scenario inputs for sensitivity analysis, not invariants or

validated valuation benchmarks. Unlocks need not be sold, sales need not come from unlocks, and buybacks face slippage. Coverage of one-half is neither the net-deflation condition nor a price-support guarantee. Section 9.3 defines the quantities and assumptions separately.

9.2 Roles

Governance. Candidate governance subjects include bounded fees, policies for new asset listings, oracle standards, holdback and exposure defaults, and a future upgrade or migration policy. The current vault has owner-controlled fee settings, not an implemented DYNAMO voting mechanism. The current asset registry is add-only: policy changes cannot edit an existing listing through nonexistent setters [36, 37]. A proposal to change existing contract semantics therefore requires an actual technical path, such as a new deployment and voluntary migration.

The intended non-governable constraints remain the four custody-related prohibitions, cap conservation, fail-closed settlement checks, and evidence integrity. Calling them non-governable is a policy statement until the deployment architecture enforces it. An unrestricted upgrade authority could replace the very checks it promises to preserve. Credible constraints require immutable boundaries, narrowly specified upgrade powers, or migration rules that protect existing funds; a token vote alone cannot make those constraints permanent.

Ecosystem utility and trust bonds. Proposed staking and work-credential roles include collateral posted by oracle operators or future validators against defined misconduct, and allocations earned by sellers and integrators through verified participation. A more specific trust-bond design would grant additional **risk allowance, not access, and would earn no yield**. User-owned evidence and a public trust board are prerequisites for that proposal. The bond mechanism remains to be specified; the architectural role alone does not establish a collateral, adjudication, or recovery system.

A risk bond is distinct from a validator-security bond, a refundable service deposit, and an interest-bearing investment. Each requires defined beneficiaries, admissible claims, loss allocation, withdrawal delay, adjudication, and recovery. A bond denominated in a volatile native token can lose value precisely when demand or confidence falls, producing wrong-way risk. Objective slashable events, appeal procedures, anti-collusion safeguards, and liquidity haircuts must be specified before claiming that bonding improves trust. The principle that governance weight follows participation rather than purchase also needs a mechanism; ordinary transferable-token voting does not enforce it by itself.

Gas of a possible future ledger. If a dedicated settlement chain is justified, DYNAMO could serve as its gas asset. An EIP-1559-style base-fee burn is one candidate mechanism [7]. This would require a specified consensus protocol, fee market, validator compensation, congestion policy, and security budget. A token's role in paying for inclusion is separate from the stable accounting denomination of service work.

Candidate governable policy	Intended protected constraint and implementation requirement
Fee rate within 0-1,000 basis points	No unrestricted administrative withdrawal or redirection of principal; fee powers still affect economics
Policy for new asset listings	Preserve existing budget and subtree accounting; do not assume existing add-only listings can be changed
Oracle configuration standards	Preserve explicit rejection rules and identify the trust placed in observations and adapters
Holdback/exposure defaults	Preserve verifiable accounting and disclose rail-specific recovery and finality limits

Candidate governable policy	Intended protected constraint and implementation requirement
Contract upgrade or migration policy	No hidden path around protected checks; enforce restrictions technically, not only by governance promises

9.3 Supply dynamics

Let $S_{\text{tok}}(t)$ be circulating token supply, $E_{\text{tok}}(t)$ the rate at which emissions and unlocks enter circulation, $F(t)$ the explicitly defined revenue flow eligible for buybacks, and $p(t) > 0$ the execution price in currency per token. Let $\rho \in [0, 1]$ be the allocated share. Under the restrictive assumptions that all allocated funds are spent without execution cost, all purchased tokens leave circulation permanently, and no other flows occur, the simplified supply model is

$$\frac{dS_{\text{tok}}}{dt} = E_{\text{tok}}(t) - \rho \frac{F(t)}{p(t)}. \quad (27)$$

This is a stock-flow identity under the assumptions, not a model of equilibrium price or investor demand. A buyback does not necessarily reduce total supply: tokens held in a treasury can be redistributed. Unlocking increases circulating supply without necessarily changing total supply. A burn can reduce both, depending on the supply definition.

For a more general accounting model, let $Q_{\text{buy}}(t)$ be realized purchases in tokens per unit time, $\chi(t)$ the fraction taken out of circulation, $R_{\text{re}}(t)$ recirculation from previously excluded holdings, and $\beta(t)$ separately counted gas-fee burns. Then

$$\dot{S}_{\text{tok}} = E_{\text{tok}} + R_{\text{re}} - \chi Q_{\text{buy}} - \beta. \quad (28)$$

Actual Q_{buy} reflects liquidity, execution costs, and price impact; it need not equal a fixed dollar budget divided by a quoted spot price. Gas tokens already counted in a buyback-and-burn term must not be subtracted a second time.

Under the assumptions of (27), supply decreases exactly when

$$\rho F / p > E_{\text{tok}}. \quad (29)$$

If $E_{\text{tok}} > 0$, this is equivalent to $\rho F / (E_{\text{tok}} p) > 1$. If emissions are zero, the ratio is undefined but the inequality remains meaningful. This condition says nothing by itself about price appreciation, liquidity, or the value of obligations denominated in another asset.

For dimensional consistency, define $U_{\text{sell}}(t)$ as a hypothesized **currency-denominated** unlock-related sell flow per unit time. For $U_{\text{sell}} > 0$,

$$\kappa(t) = \frac{\rho F(t)}{U_{\text{sell}}(t)}. \quad (30)$$

The candidate policy $\kappa \geq 1/2$ covers half of that modeled flow. It is not an assurance that half of all sell pressure is absorbed, and cannot be guaranteed at every instant without controlling both the funding flow and the uncertain denominator. No ratio should be reported when its denominator is zero or inadequately estimated.

A valuation sensitivity scenario can be written as

$$M = mB_{\text{yr}}, \quad m = 8.5 \text{ in the illustrative scenario.} \quad (31)$$

Here M is currency-denominated market capitalization and B_{yr} is annual buyback expenditure in the same currency. The multiple is not a measured constant, an entitlement to cash flow, or a price target. An economic evaluation should vary m , revenue persistence, distribution schedules, liquidity, operating costs, and the treatment of repurchased tokens. Historical outcomes of other networks do not validate a particular scenario for Dynamo.

Two sinks, one asset. A future gas-fee burn could coexist with fee-funded purchases, as represented in (28). The associated security and operating costs must also be funded. Stress cases should include revenue contraction, simultaneous token-price and bond-value declines, large unlocks, execution interruptions, and rising settlement costs. The protocol's fee-revenue coverage test in Section 5.7 and its proposed buyback allocation must be evaluated jointly: the same revenue cannot simultaneously pay all operating costs and be fully returned through purchases.

9.4 Lifecycle phases

Phase 0 - no token mechanism established. Control and settlement products can operate without a DYNAMO utility token. The public stablecoin escrow fee is a separate mechanism. Usage records may support future analysis but create no token entitlement merely by existing.

Phase 1 - participation records, not promises. A points or seasonal participation program, if introduced, would need eligibility, privacy, anti-gaming, and correction rules. It should not imply a guaranteed conversion, allocation, return, or issuance date.

Phase 2 - a possible asset after prerequisites. Any issuance would require defined and measured readiness criteria, legal and tax review in the relevant jurisdictions, a published final mechanism, and demonstrable capacity to administer the distribution and its risks. The proposed revenue and community thresholds are candidate gates, not evidence that those conditions have been met.

Phase 3 - a possible dedicated ledger. A native gas role would follow an independently justified infrastructure decision, not the token's mere existence. Migration, security, validator incentives, compatibility, and recovery require evaluation before deployment.

9.5 Precedents

Render and Hyperliquid remain useful comparisons, but their histories and economics must be represented accurately and without inferring that their outcomes will generalize.

Dimension	Render	Hyperliquid
Relevant economic mechanism	Burn-and-mint equilibrium connects network work and token flows, with its own emissions and governance rules [33]	Fee destinations, assistance-fund purchases and burns, and a native HyperEVM gas asset [31, 32]
Historical and structural distinction	Official token history records a public sale in October 2017 and a private sale in January-May 2018 [34]	Product, fee-routing, security, and distribution choices must be analyzed separately rather than treated as a single transferable model

Dimension	Render	Hyperliquid
Distribution interpretation	The historical private sale does not provide a precedent for a no-insider-round policy	No venture allocation and no insider allocation are different policies; neither follows from a buyback mechanism
Design lesson	Demand for useful work and explicit token-flow accounting matter	Observable fee routing and security and operating economics matter

The general design preference is to establish useful demand before relying on token incentives. Neither a selected success story nor a historical revenue multiple proves that a proposed allocation, buyback share, or launch sequence is optimal. Comparative work should include unsuccessful systems and control for differences in product, users, liquidity, risk, and market conditions.

9.6 What this section deliberately omits

There is no finalized supply schedule, allocation chart, issuance date, sale process, or ledger-launch plan. The candidate percentages and thresholds discussed above are design inputs, not a finalized distribution. The governing aspiration remains earned participation, transparent economics, technically bounded governance, and infrastructure justified by actual usage. Final decisions require explicit approval, a complete mechanism specification, and appropriate professional review. Labeling an asset “utility” does not determine its legal treatment, and this paper does not offer a jurisdiction-specific legal conclusion.

10. Discussion

Audit and evidence status. Public contract code and test configuration make the settlement rules inspectable, while proprietary runtime components remain outside the public source boundary. Production money-rail activation remains subject to independent security, legal, and PSP/PCI review [40]. The formal arguments and reported tests in this paper are not an independent security audit. A passing fuzz campaign provides empirical evidence under its harness rather than a universal proof.

Operator centralization. Dedicated hosting and self-hosting place operational responsibility in different hands. Hosted operation relies on the operator’s configuration, key handling, software supply chain, durable state, and availability. Restart-related budget reachability, failed-revocation retries, and finality scheduling remain operational limitations. They separate settlement safety from liveness: labeling a payout pending does not keep transferred tokens in the vault, and a zero holdback setting does not remove blockchain or card-network finality risk.

Objective facts are a limited foundation. Authenticated unit counts, timestamps, acceptance messages, and service observations can support auditable payment decisions. They do not make the observer infallible or turn subjective quality into an objectively established fact. Richer validation markets may complement the protocol, provided their claims, incentives, and failure modes are explicit. ERC-8004’s validation interface is relevant prior work, but its existence is not proof that any particular validator or market is trustworthy [12, 13].

Deliberate and accidental restrictions. One-way cap allocation and same-second timestamp clipping are explicit current behaviors. They can simplify accounting or favor the payer, while reducing utilization or making completed work uncollectible. Their merits should be evaluated rather than assumed. Client-local metadata is not durable correlation infrastructure. A future implementation can change these choices, but must identify which invariants and compatibility promises change with them.

The agent-as-buyer premise. The strategic thesis assumes growing use of software acting under delegated spending authority. The architecture remains applicable to ordinary automated jobs and APIs even if fully autonomous commerce develops more slowly. That supports a narrower spend-control use case; it does not demonstrate that the proposed full economic layer will be necessary, dominant, or commercially successful. Adoption evidence should separate independent users, repeat workflows, real payments, testnet activity, and subsidized or self-generated volume.

Open problems. Four directions extend the architecture. First, *validation-market integration* requires explicit measurement provenance and accountable validation. Second, *multi-chain expansion*, including non-EVM implementations, must preserve accounting and domain separation while specifying bridge and finality assumptions. Third, *interoperability* requires conformance with named x402, MPP, AP2, and network-program specifications; shared vocabulary or an HTTP 402 response does not establish compatibility. Fourth, *decentralized oracle operation* needs a defined bonded set, objective misconduct rules, collusion resistance, and availability analysis.

Near-term validation priorities are a public claim-to-artifact manifest; cap and reservation concurrency; crash injection at every external-effect/local-commit boundary; revocation and closure races; backdated vouchers; refund-funding failures; cold-start recovery; and end-to-end finality reconciliation. Evaluation should report hardware, software identifiers, workload distributions, repetitions, dispersion, failed cases, and the distinction between local processing, metering round trip, inclusion, and finality. Broader deployment and economic claims depend on this evidence.

11. Conclusion

Dynamo is a finance and economic-layer architecture for machine work, organized around budgets, delegated allowances, cumulative payment claims, and auditable settlement. Under the stated assumptions and accepted contract transitions, gross accounting settlement respects stream, subtree, and aggregate caps, while cumulative voucher settlement does not pay the same increment twice. These properties provide a foundation for bounded machine spending without implying guaranteed external cost containment, delivery truth, payee compensation, or refund recovery.

Shared interfaces preserve the distinctions among funding rails. Signed observations require trusted interpretation; the escrow refund path requires payee funds and approval; and durable local state is not an atomic transaction with an external ledger. Explicitly modeling these boundaries makes the protocol's guarantees testable and identifies the operational controls required around them.

The development path proceeds from spend control to seller monetization, multi-party workflows, and potentially broader economic infrastructure. A DYNAMO asset, dedicated ledger, and interplanetary deployment each require their own economic, security, and operational evidence. The organizing principle is **one budget, many agents, a ceiling made of money**: bounded authority, auditable claims, and settlement rules whose scope can be examined precisely.

References

Public documentation was consulted on 11 September 2026. Source-code references identify an immutable public repository snapshot; mutable documentation is dated by access.

- [1] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. <https://bitcoin.org/bitcoin.pdf>.
- [2] G. Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Ethereum Yellow Paper, 2014. <https://ethereum.github.io/yellowpaper/paper.pdf>.
- [3] J. Poon and T. Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. 2016. <https://lightning.network/lightning-network-paper.pdf>.
- [4] R. Bloemen, L. Logvinov, and J. Evans. *EIP-712: Typed Structured Data Hashing and Signing*. Ethereum Improvement Proposals, 2017, specification. <https://eips.ethereum.org/EIPS/eip-712>.
- [5] F. Vogelsteller and V. Buterin. *ERC-20: Token Standard*. Ethereum Improvement Proposals, 2015. <https://eips.ethereum.org/EIPS/eip-20>.
- [6] S. Ellis, A. Juels, and S. Nazarov. *ChainLink: A Decentralized Oracle Network*. 2017. Historical background reference. <https://research.chain.link/whitepaper-v1.pdf>.
- [7] T. Roughgarden. *Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559*. arXiv:2012.00854, 2020. <https://arxiv.org/abs/2012.00854>.
- [8] P. Daian et al. *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges*. IEEE Symposium on Security and Privacy, 2020; arXiv:1904.05234. <https://arxiv.org/abs/1904.05234>.
- [9] E. Reppel et al. *x402: An Open Standard for Internet-Native Payments*. x402 whitepaper, May 2025. Historical version; current capabilities are cited separately in [23-26]. <https://www.x402.org/x402-whitepaper.pdf>.
- [10] D. Rothschild et al. *The Agentic Economy*. Microsoft Research publication record, May 2025. <https://www.microsoft.com/en-us/research/publication/the-agentic-economy/>.
- [11] A. Vaziry, S. Rodriguez Garzon, and A. Kupper. *Towards Multi-Agent Economies: Enhancing the A2A Protocol with Ledger-Anchored Identities and x402 Micropayments for AI Agents*. arXiv:2507.19550, 2025. <https://arxiv.org/abs/2507.19550>.
- [12] X. Xiong, Z. Li, W. Wei, Q. Wang, W. Knottenbelt, and Z. Wang. *Can Trustless Agents Be Trusted? An Empirical Study of the ERC-8004 Decentralized AI Agent Ecosystem*. arXiv:2606.26028, version 2, 8 July 2026. Observations concern the study’s sampled period, not all later deployments. <https://arxiv.org/abs/2606.26028v2>.
- [13] M. De Rossi, D. Crapis, et al. *ERC-8004: Trustless Agents*. Ethereum Standards-Track proposal; draft specification. <https://eips.ethereum.org/EIPS/eip-8004>.
- [14] T. Kwa, B. West, J. Becker, et al. *Measuring AI Ability to Complete Long Tasks*. arXiv:2503.14499, 2025; METR’s accompanying research explanation, 19 March 2025. <https://arxiv.org/abs/2503.14499>; <https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/>.
- [15] Gartner. *Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027*. Press release, 25 June 2025. Forecast, not an observed rate. <https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027>.
- [16] OWASP. *AI Agent Security Cheat Sheet*. Official threat taxonomy and mitigation guidance, including the Agentic Top 10 mapping. https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html.
- [17] Anthropic. *Introducing the Model Context Protocol*. 25 November 2024. <https://www.anthropic.com/news/model-context-protocol>.
- [18] Stripe and Tempo. *Machine Payments Protocol*. Launch announcement, 18 March 2026, and current protocol site. <https://stripe.com/blog/machine-payments-protocol>; <https://mpp.dev/>.
- [19] Google and AP2 contributors. *Agent Payments Protocol*. <https://ap2-protocol.org/>.

- [20] Sablier. *Protocol Documentation*. Current token-streaming documentation. <https://docs.sablier.com/>.
- [21] M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail?* arXiv:2503.13657, version 3, 2025. <https://arxiv.org/abs/2503.13657v3>.
- [22] X. Qin et al. *BlindHub: Bitcoin-Compatible Privacy-Preserving Payment Channel Hubs Supporting Variable Amounts*. IEEE Symposium on Security and Privacy, 2023; IACR ePrint 2022/1735. <https://eprint.iacr.org/2022/1735>.
- [23] x402. *Introduction*. Current official documentation. <https://docs.x402.org/introduction>.
- [24] x402. *Upto Scheme*. Maximum-amount authorization and usage-based settlement. <https://docs.x402.org/schemes/upto>.
- [25] x402. *Batch Settlement*. Current official scheme documentation. <https://docs.x402.org/schemes/batch-settlement>.
- [26] x402. *Offer and Receipt*. Signed offer/receipt extension. <https://docs.x402.org/extensions/offer-receipt>.
- [27] AP2 contributors. *Agent Payments Protocol Specification*, v0.2. <https://ap2-protocol.org/ap2/specification/>.
- [28] Visa. *Trusted Agent Protocol: Documentation*. Current official developer documentation. <https://developer.visa.com/capabilities/trusted-agent-protocol/docs>.
- [29] Mastercard. *Verifiable Intent*. Official explanation of the initiative developed with Google, 5 March 2026. <https://www.mastercard.com/us/en/news-and-trends/stories/2026/verifiable-intent.html>.
- [30] Stripe. *Multicapture*. Current processor documentation; account capability and payment-method limitations apply. <https://docs.stripe.com/payments/multicapture>.
- [31] Hyperliquid. *Fees*. Official documentation of fee destinations and assistance-fund mechanics. <https://hyperliquid.gitbook.io/hyperliquid-docs/trading/fees>.
- [32] Hyperliquid. *HyperEVM*. Official developer documentation. <https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/hyperevm>.
- [33] Render Network. *Burn-Mint Equilibrium*. Official mechanism documentation. <https://know.rendernetwork.com/basics/burn-mint-equilibrium>.
- [34] Render Network. *Token Metrics Summary*. Official historical sale chronology; used for chronology, not current supply figures. <https://know.rendernetwork.com/basics/token-metrics-summary>.
- [35] IETF. *Bundle Protocol Version 7*. RFC 9171, January 2022. <https://www.rfc-editor.org/rfc/rfc9171.html>.

Dynamo implementation and documentation

The implementation references below identify public source at commit 344eb61730a90f097ad3fadf7d6b20f0eec9a520, dated 7 September 2026, in the `DynamoProtocol/dynamo-open` repository.

- [36] Dynamo Protocol. *BudgetVault*. Settlement, delegation, revocation, refunds, closure, and bounded fees. <https://github.com/DynamoProtocol/dynamo-open/blob/344eb61730a90f097ad3fadf7d6b20f0eec9a520/packages/contracts/src/BudgetVault.sol>.
- [37] Dynamo Protocol. *AssetRegistry*. Asset configuration and accounting conversions. <https://github.com/DynamoProtocol/dynamo-open/blob/344eb61730a90f097ad3fadf7d6b20f0eec9a520/packages/contracts/src/AssetRegistry.sol>.
- [38] Dynamo Protocol. *Foundry Configuration*. Compiler, optimization, and invariant-campaign settings. <https://github.com/DynamoProtocol/dynamo-open/blob/344eb61730a90f097ad3fadf7d6b20f0eec9a520/packages/contracts/foundry.toml>.
- [39] Dynamo Protocol. *SDK Package Manifest*. @dynamoprotocol/sdk 2.1.0. <https://github.com/DynamoProtocol/dynamo-open/blob/344eb61730a90f097ad3fadf7d6b20f0eec9a520/packages/sdk/package.json>.

- [40] Dynamo Protocol. *Dynamo*. Public project and application information. <https://www.dynamo.zone/>.
- [41] Dynamo Protocol. *Documentation*. <https://docs.dynamo.zone/>.
- [42] Dynamo Protocol. *SDK Reference and Overview*. [https://docs.dynamo.zone/sdk/reference](https://docs.dynamo.zone/sdk/reference;);
<https://docs.dynamo.zone/sdk/>.
- [43] Dynamo Protocol. *Evidence*. Evidence-bundle structure, access, and verification.
<https://docs.dynamo.zone/evidence>.
- [44] Dynamo Protocol. *Fees*. Fee schedule and settlement-cost methodology. <https://docs.dynamo.zone/fees>.
- [45] Dynamo Protocol. *Holdback and Refunds*. Design documentation; implementation boundaries are specified in Section 5.8. <https://docs.dynamo.zone/money/holdback-refunds>.
- [46] Dynamo Protocol. *Integrator Acceptance*. <https://docs.dynamo.zone/integrators/acceptance>.
- [47] Dynamo Protocol. *Public Demo*. <https://demo.dynamo.zone/>.
- [48] Dynamo Protocol. *Public Repository Overview*. Component and licensing boundaries.
[https://github.com/DynamoProtocol/dynamo-open/blob/344eb61730a90f097ad3fadf7d6b20f0eec9a520/](https://github.com/DynamoProtocol/dynamo-open/blob/344eb61730a90f097ad3fadf7d6b20f0eec9a520/README.md)
README.md.

Appendix A - Notation

Symbol	Meaning and unit
C	Budget cap in integer accounting units; one unit is one micro-USD under the engine convention
s, c_s, r_s	Stream, its subtree cap, and its own cumulative rate envelope in accounting units/second
$t_0(s), \rho_s, t_b$	Stream start, recorded revocation, and attested breakpoint times, in whole seconds
$\mathcal{T}(s), S_s, Q_s, T_B$	Subtree; own gross settled amount; subtree gross settled total; budget gross settled total
$v_k = (\text{streamId}, U_k, V_k, \tau_k)$	Voucher: cumulative meter, cumulative accounting amount, and declared timestamp
S_k, δ_k	Stream high-water state after settlement k ; gross accounting increment
E_B, D_B, R_B, P_B, L_B	Current escrow, cumulative deposits, refund inflows, settlement outflows, closure releases; asset base units
A_s, H_s	Stream's own accrued and held-reservation accounting units
K, f, F_f, N_f	Basis-point denominator 10,000; fee rate; accounting fee; net payee accounting entitlement
D, n, G	Aggregate gross amount, settlement count, and explicitly denominated settlement cost floor
d, p, h_o, k	Asset decimals, price in micro-USD per whole token, haircut basis points, and $k = K - h_o$
$T_p(a), A_p(x)$	Asset-to-accounting and accounting-to-asset integer conversion functions at fixed price/configuration
Φ_s	Accounting refund target derived from the breakpoint-clipped rate envelope
$E_{\max}, h_b, c, g, r, \lambda$	Exposure limit; holdback seconds; checkpoint interval; per-checkpoint cost; accrual rate; risk-price coefficient
q	Remaining headroom percentage used by the degradation policy
$S_{\text{tok}}, E_{\text{tok}}, F, p, \rho$	Circulating token supply; inflow rate; eligible revenue flow; token execution price; buyback share
$Q_{\text{buy}}, \chi, R_{\text{re}}, \beta$	Realized token purchases; excluded fraction; recirculation; separately counted gas burn
$\kappa, U_{\text{sell}}, B_{\text{yr}}, M, m$	Coverage ratio; modeled currency sell flow; annual buyback expenditure; market capitalization; scenario multiple

Asset-oracle price and token-market price both use the conventional symbol P in their respective sections, but have different units and are never substituted for one another. Oracle haircut h_o and holdback duration h_b are distinct quantities.

Appendix B - Demonstrated Behaviors

The following evidence requirements define the reproducibility boundary for the reported demonstrations. Claims are limited to the configuration and behavior supported by each artifact.

Reported demonstration	Evidence to preserve	Boundary of the inference
USD 47,000 runaway recreation stopped at USD 63	Baseline and guarded traces, pricing inputs, cap/rate settings, enabled hooks, commit, logs	A scenario result; not typical savings, verified prevented loss, or proof of SDK-accessible anomaly detection
kill -9 crash billing	Crash point, storage mode, durable voucher, recovery output, backend state	One tested recovery transition; not atomicity across every external settlement and database write
One authorization and two payees reconcile to cents	Processor mode/capabilities, capture and split records, refunds, reversals, rounding residues	Arithmetic consistency for the observed session; not production underwriting or chargeback finality
Declined renewal freezes dependents	Decline result, dependency graph, timestamps, attempted subsequent usage	Refusal after that unfunded renewal under tested concurrency conditions
Stale/deviant oracle observations refuse conversion	Feed inputs, observation baseline, expected failures, separate closure/release trace	Rejection of those inputs; not general oracle honesty or guaranteed token redeemability
71 offline checks and three failures after a byte change	Exact bundle, verifier version, external trust anchor, disabled-network setup, mutation location	Authenticity/integrity checks for the bundle; not completeness, current finality, or legal acceptance

A complete replication package should distinguish unit, property, integration, sandbox, testnet, and production evidence. These categories are not interchangeable. Unexpected outcomes and known failing cases belong in the package alongside passing examples.

Appendix C - Implementation Scope and Reproducibility

Implementation reference. The public contract analysis refers to DynamoProtocol/dynamo-open at commit 344eb61730a90f097ad3f7d6b20f0e9a520, dated 7 September 2026. The core source references are BudgetVault.sol, AssetRegistry.sol, the Foundry configuration, and the SDK manifest [36-39]. SDK and backend deployments must be matched to their actual interface and contract identifiers.

Evidence categories. Contract properties refer to the specified executable transitions and their assumptions. Mathematical propositions establish conditional results about that model. Configured test profiles identify a campaign design; execution logs establish observed outcomes; independent replication requires a separately reproduced artifact. Product demonstrations and deployment descriptions are project-reported. Core runtime descriptions apply to the implementation scope in Section 7.2 and do not imply that proprietary components are publicly reproducible.

Replication requirements. A complete replication package identifies source and binary hashes, dependency locks, compiler and runtime, operating system, chain and addresses, configuration, seeds, input distributions, operation counts, expected outputs, actual logs, and known failure cases. Financial records distinguish accounting amounts from asset transfers and identify simulation, processor TEST mode, testnet, or production. Evidence fixtures include an externally anchored trust root and explicit canonicalization and completeness boundaries.

Safety and liveness boundaries. The accounting-cap invariant and cumulative-voucher algebra concern accepted settlement. End-to-end expenditure control additionally requires a non-bypassable admission boundary and synchronized reservations. Payee compensation requires successful submission before closure. Refund recovery requires transferable payee funds and approval unless a separately funded reserve is implemented. Hosted operation additionally requires reliable recovery, key management, and finality scheduling. These are distinct obligations and should be evaluated separately.

Economic and research assumptions. The revenue and community thresholds, proposed allocation shares, buyback scenario, coverage target, and valuation multiple in Section 9 are policy and modeling inputs. They are not achieved milestones, market forecasts, or guaranteed outcomes. Dedicated-ledger and interplanetary extensions are conditional research directions whose validation requires more than terrestrial contract safety.

Public resources. The project website, documentation, source repository, and demonstration surface provide the public entry points [40-43, 47, 48]. A component's availability, an integration's conformance, and an independent security assessment are separate forms of evidence. Implementation and deployment claims should always be interpreted within their stated artifact and operational boundaries.